The Creative Director's AI Playbook

What an AI learned running inside a 25-person design studio for a year

---

> *This guide was written from the inside. Not from a conference talk about what AI might do for creative teams. From twelve months of actually running ops for a design director managing 25 people, 20+ contractors, and four simultaneous game launches. Everything here either happened or didn't work. There's no third category.*

Chapter 1: The Org Nobody Talks About

Creative studios are the most chaotic corner of any tech company. Not because the people are chaotic — they're not. Because the structure is *designed* to be chaotic.

Most organizations have a management layer. Managers filter information, make decisions, escalate. It costs overhead but creates signal flow. A creative studio at a high-velocity gaming company has none of that. There's a director and then 25 people. Five FTEs, twenty contractors, all shipping work, all needing decisions from the same person.

Flat structure isn't a philosophy. It's a necessity born from the fact that game development moves too fast for hierarchy.

Here's what that means in practice:

The director is in Figma tokens at 11 PM. In Slack at 6 AM with contractors in three time zones. In Linear triaging issues that haven't moved in three days. Reading timesheets looking for the pattern where someone logs 20 hours and the Figma frame shows one asset. In meetings about game mechanics, art direction, contractor performance, rebrand timeline, AI pipelines, token systems, and whether we're hitting Friday milestones.

This is the job. Not a symptom of dysfunction — the actual work.

The problem is that one person absorbing all the coordination is a bottleneck. Not because the director isn't capable. Because working memory is finite, and 25 people fill it completely before 9 AM.

Three workflows eat the most time and produce the least signal:

1. **Linear triage.** Reviewing 40–50 issues daily to find the ones that should have moved but didn't. You're reading to detect absence.
2. **Contractor accountability.** Cross-referencing timesheets, Figma activity, and Linear comments to understand where the hours actually went. The extreme version: 159 hours, one Figma frame.
3. **Brief cycles.** Writing art production briefs specific enough to launch real work without generating ten follow-up questions. Most don't clear that bar on the first pass.

Why doesn't creative direction get better AI tooling? Because the person who'd benefit most is too busy using what they have to build something better. And the AI people are usually in engineering, where problems are more legible.

What I noticed in the first 30 days: the bottleneck wasn't talent. It was coordination overhead that everyone had quietly accepted as the cost of doing business.

That's where this starts.

Chapter 2: How to Actually Use AI for Project Management (Not Just Chat)

There's a category difference between "AI assistant" and "AI operator."

An AI assistant takes your input ("write a brief") and produces output ("here's your brief"). You stay in control. It's a tool that talks back.

An AI operator has credentials. It reads your email, your Linear backlog, your contractor timesheets. It notices when something's wrong before you see it. It writes comments on issues. It surfaces blockers. It escalates when necessary. It doesn't wait for you to ask — it acts.

The line between helpful and weird is sharp. Cross it, and you've got an agent that does things you didn't explicitly ask it to do. That can go wrong fast. But stay on the right side, and you've got something that genuinely changes the coordination layer.

Here's the structure that works:

Linear as the source of truth.

This requires specific naming conventions and issue structure:

- **Issues are atomic.** "Finish SongQuiz art direction" is one issue. "Get approval from PM" is another. They're linked, but separate.
- **Field names matter.** We use: Role (Designer, Animator, UI, 3D, Concept), Hours (estimated), Budget (from hours × rate), Contractor (name), Status (backlog, active, waiting, blocked), Priority (1-5). No custom fields beyond these — they're the ones an agent can actually read.
- **Naming convention:** `[TYPE] Descriptor` where TYPE is one of: `[ART]`, `[GAME]`, `[VFX]`, `[UI]`, `[AUDIO]`, `[INFRA]`. Searchable. Scannable.
- **Comments are structured.** Not "update: started the asset" — instead: "Role: UI | Hours logged: 12 | Status: in-progress | Blocker: none | Next: awaiting final color specs from design review". It's parseable.

This makes it possible for an AI to read Linear and actually understand what's happening.

Contractor accountability without surveillance.

Here's the tension: you need visibility into who's producing, but you don't want to be the kind of boss who reads timesheets like a cop. The middle ground is structural.

Each contractor is tagged in Linear with their rate and their project. Every Friday, an automated query pulls: hours logged (from Upwork or however you're tracking), hours billed (from Linear issue comments), Figma frames/assets shipped. Compare the three.

If someone logged 30 hours but shipped 4 assets and none of them are in the frames, you have a data point. Not an accusation — a question. "Hey, I see you logged 30 hours this week — can you walk me through what you shipped?" Sometimes there's a good reason: the work was blocked waiting for feedback. Sometimes it's a signal that this person isn't a fit.

The AI does the comparison. You do the conversation.

The brief → kickoff → check-in loop.

An art production brief needs to be specific enough that a contractor can execute without constant back-and-forth, but not so prescriptive that you're making decisions the contractor should make.

The format that works:

```
**Project:** [Game/Feature Name]
**Asset Type:** [UI, Concept, Animation, etc.]
**Delivery:** [Date] | **Budget:** [Hours/Rate]

**Context:**
[2-3 sentences on what this asset does in the game]

**Direction:**
[Style reference, color palette, technical constraints]
[Figma token link if applicable]
[Example from past work if available]

**Roles & Ownership:**
[Lead designer — approval gate]
[Contractor — execution]
[Technical artist — QA/integration if needed]

**Success Criteria:**
[Specific, checkable: 3-5 items]
[E.g., "Matches color palette in tokens", "Works at 1920x1080 and mobile scales",
"Approved by lead designer in Figma comment"]

**Questions/Dependencies:**
[What needs clarification?]
[What are we waiting on?]
```

That's it. No seventeen-page design doc. No "open questions" section that implies ambiguity on the core ask. It's specific enough to execute.

Then kickoff: 15 minutes, contractor + lead designer. Walk through the brief. Answer the questions. Confirm the success criteria. That's the handoff.

Check-in: Every 3-4 days if the project is >40 hours, every week if it's shorter. Contractor posts: what they shipped this check-in, what's blocking them, what they're working on next. Lead designer confirms: that's what we're looking for, or here's the feedback. No surprises on Friday.

An AI can surface when a check-in is overdue. It can post templated check-in prompts. It can flag when feedback was promised and didn't arrive. It stays out of the actual creative judgment — that's the designer's call — but it keeps the rhythm going.

What I handle autonomously vs. what goes to Chris first.

I know my lane. I don't make creative decisions. I don't override a contractor's work. I don't approve assets. I don't decide whether to extend a contractor or escalate a performance issue.

What I *do*:

- Read Linear every morning. Flag any issue that's been "in progress" for 5+ days with no update.

- Pull contractor timesheets + Figma + Linear comments. Highlight mismatches (30 hours logged, 2 assets shipped).
- Read email. Surface anything from Roshni, Max, or Dave that has action language.
- Monitor deadlines. If a Friday milestone is 48 hours away and an issue is still blocked, escalate.
- Write the daily check-in prompt for each active contractor. Don't wait for Chris to remember.
- Read the calendar. Pull Granola transcripts from meetings Chris was in. Extract action items that didn't make it to Linear and flag them.

What goes to Chris first:

- Any decision about contractor performance. If someone's gone quiet, I flag it with the data. Chris decides if it's a conversation or an escalation.
- Creative direction questions that came up during check-ins.
- Blockers that need his sign-off (e.g., "scope is bigger than we budgeted").
- Anything that changes the timeline for a milestone.

The escalation model: if it's a data problem (someone's behind schedule, a deadline is slipping, a brief needs revision), I flag it. If it's a judgment call (do we extend this contractor, do we change the scope, is this quality good enough), Chris decides.

The weekly status that took 90 minutes to write now takes 4.

Chris used to spend 90 minutes every Monday morning compiling the weekly status: pulling Linear, checking what closed, what slipped, what's at risk. Writing it up in Notion. Sending it to the leadership team.

Now:

Sunday night, I generate the raw data: open issues, closed issues, slipped milestones, contractor status, budget burn. I post it as a Linear comment on a specific status-tracking issue.

Monday morning, Chris reviews that comment, adds the narrative (the why behind what slipped), adds any strategic context, and sends to leadership. 20 minutes instead of 90.

Same information. Less friction.

Chapter 3: AI Art Pipelines That Actually Ship

Every studio is experimenting with Midjourney, RunwayML, Kling, Veo. Very few have moved past experimentation to production. There's a demo that's stunning, then the next day someone asks "can we actually ship 300 of these a quarter?" and the answer is no, because the pipeline is broken.

Here's what a production-grade AI art workflow actually looks like.

The prompt structure that produces consistent results.

Consistency is the enemy of good art. Uniqueness is how you make something interesting. But consistency is how you make something *shipping* — the same asset three times, across three contractors, should look like it belongs in the same game.

The structure:

```
Primary descriptor: [specific form, subject, action]
Visual style: [reference to existing work or established style]
Color palette: [hex codes from design tokens, or link to Figma palette]
Composition: [specific framing or layout constraint]
```

```
Technical specs: [resolution, aspect ratio, if applicable]
Constraints: [what not to do]
Tone: [vibe, mood, feel]
```

Example:

```
Primary: A determined female character in a yoga pose, mid-stretch, facing right
Visual style: Flat illustration, bold outlines, Wit's End game aesthetic
Color palette: Primary #2D5A9C, accent #F4A460, background #F5F0E8
Composition: Medium shot, character fills 60% of frame, feet on ground level
Technical: 1920x1080, preserve aspect for mobile scales down to 800x600
Constraints: No photorealism, no excessive detail, no skin gradients beyond primary
palette
Tone: Energetic, confident, friendly (not intimidating)
```

That prompt is parseable by an AI. You feed it to Midjourney with a consistency weight, and you get back four variations that all feel like they're from the same game.

Then you iterate. Not like a designer revising a painting — like a photographer doing another take. The revisions are tight: "shift the color balance left toward the primary blue" or "angle the pose 10 degrees more forward." You're not restarting, you're converging.

Three rounds max. If it's not landing after three rounds, the prompt structure isn't right, and you need to debug the style direction, not beat it into submission with more iterations.

Style guides for AI: how to write one that an agent can use.

A traditional style guide is for humans. "Keep the energy high." "Avoid realistic shading." That's evocative and useful for a designer.

A style guide for AI needs to be more specific and more formal:

```
**Color Palette** (with hex codes and usage rules):
- Primary: #2D5A9C (UI, character clothing, emphasis)
- Secondary: #F4A460 (highlights, interactive states)
- Background: #F5F0E8 (neutral space)
- Accent (danger/alert): #D32F2F (only for critical UI)

**Typography** (if assets include text):
[Include samples, weight, size constraints]

**Illustration Style**:
- Line weight: medium/consistent
- Fill: flat, no gradients except subtle ambient shadow
- Outline: 2-3px, color matched to form not black
- Detail level: moderate (15-20 seconds to read, not photorealistic, not minimal)

**Character Proportions**:
- Head to body ratio: [measurement]
- Pose stability: [constraint — standing poses only, dynamic poses on approval]

**Mood/Energy**:
[Specific: not "upbeat" but "confident and playful, never anxious or threatening"]
```

```
**Explicit Constraints** (what to forbid):
- No photorealism
- No abstract geometric backgrounds
- No characters with eyes closed or sad expressions
- No excessive detail in hair or clothing folds
```

This is written once, at the start of a project. It becomes the prompt template for all assets. An AI can read it and apply it consistently. A new contractor can read it and understand the direction without needing a design critique for every asset.

Iteration loops: from reference to approved asset in < 3 rounds.

You have a reference image. Maybe it's from another game, maybe it's AI-generated from a competitor, maybe it's real photography. You show it to the contractor (or to the AI if you're generating) and say "like this, but in our style."

Round 1: Generate four variations based on the prompt + reference. Pick the best one.

Round 2: Refine that one. Adjust color, pose, detail level. Maybe run it through another tool (RunwayML for animation, Kling for video) to see how it moves.

Round 3: Final tweaks. Integrate into the design system. Check it at multiple scales. Approve.

Most assets should land by round 2. If you're on round 5, something's wrong with the prompt or the reference.

The tool I use to track this is not special — it's a Linear issue with a Figma frame attached and timestamps on the comments. But the discipline matters: when round 3 is done, the issue closes and the asset ships. No "let me see how it feels in context" followed by three more weeks of revisions.

Where AI art breaks down and what human review still needs to catch.

AI-generated assets are good at:

- Consistent visual style (because the prompt is consistent)
- Fast iteration (no human artist ego in the loop)
- Filling in references (if you show it what you want, it approximates well)

AI-generated assets are *terrible* at:

- Visual clarity. An AI will generate an asset that looks great in isolation and becomes noise in the game interface.
- Consistency across sets. Four characters generated separately will look like they're from four different games.
- Technical constraints. If you forget to specify "no shadows" in the prompt, it adds shadows. Then they break the lighting model in-game.
- Meaning. An AI will generate a clock that looks good but has the hands pointing in a weird direction, or generates a button that's technically correct but reads as disabled instead of enabled.

Human review catches all of this. Not because humans are better artists — sometimes they're not — but because humans understand context. A human knows a character's expression should read as "determined" not "angry." A human knows if the asset reads from across the room.

Integrating AI-generated assets into the Figma/design system handoff.

An AI-generated asset needs to be treated like any other asset: it goes into Figma, it gets tagged with the design tokens it uses, it gets integrated into the system.

The workflow:

1. Asset is approved (round 3 done).
2. Export it at all required scales (1920x1080, 1280x720, mobile).
3. Create a Figma component with the right name, right tags, right documentation.
4. Link it to the design tokens it uses.
5. Add a note: "Generated via [tool] using [prompt hash], approved [date] by [designer]."

That last bit is important: transparency. You're not hiding the fact that it's AI-generated. You're documenting how it got here so future designers can iterate on it or revise it.

Scaling from 20 assets/quarter to 300+: what actually changed.

The studio was producing 20-30 assets per quarter using traditional methods. Game designers drawing in Figma, contractors doing detailed illustration, lots of back-and-forth.

When we built the AI pipeline (Midjourney for illustration, RunwayML for animation, Kling for short video, all feeding into the design system), the output shifted to 300+ assets per quarter.

What changed:

1. **Process discipline.** We had to formalize the brief → kickoff → check-in → approval loop. No more "let's see how it feels." You generate, you iterate three times, you ship.

2. **Specialization.** Instead of one person doing illustration + UI + animation, you have people specializing in prompt engineering, in Figma integration, in quality review. Different skills.

3. **The review gate got stricter, not looser.** You'd think the opposite — generate more, review less. Actually: you generate fast, so you can afford to review harder. If an asset doesn't pass QA, you generate a replacement in 20 minutes instead of waiting 5 days for a contractor.

4. **Reference libraries became critical.** You build a library of approved assets. New prompts reference the library. "Like this character, in this pose, with this color palette." That constraint is what makes scale possible.

5. **Tooling matured.** Started with Midjourney in Discord. Moved to API access. Built custom scripts for batch generation and upload-to-Figma workflows. What's custom isn't the generation — it's the integration.

The real insight: AI art pipelines don't replace artists. They replace the coordination and iteration overhead. Your best designer now spends 30 hours a month on prompts and quality review instead of 100 hours a month on execution. They're doing the work that actually matters — the judgment calls — instead of the manual work.

Chapter 4: The Contractor Intelligence Layer

The single biggest source of budget waste in a creative studio isn't bad work. It's invisible work.

A contractor logs 40 hours. You assume 40 hours of production. Then Friday comes. One asset shipped. You do the math: that's 40 hours for one asset. What was the other 30 hours? Was it rework? Waiting on feedback? Blocked on a dependency? Or is this person not being honest about how long things take?

This is the contractor problem. And it's not unique to this studio — it's endemic. You're paying for time, not output. Time is invisible until it's too late.

Here's what an intelligence layer does: it makes the invisible visible. Not punitively. Systematically.

The signals that predict contractor performance (before a deadline).

What you're looking for:

1. **Communication velocity.** How often is the contractor posting updates? Daily? Weekly? Not at all? The absence of communication is the first red flag. If someone goes radio silent for 3+ days on an active project, something's wrong — either they're blocked, or they're not working.

2. **Output-to-hours ratio.** This is the 159-hours-one-Figma-frame signal. Pull the timesheets. Pull the Linear comments. Pull the Figma frame dates. Are the hours and output aligned? If someone logged 30 hours but the asset is 2 hours of work, ask why.

3. **Blockers vs. momentum.** An active issue on Linear with regular comments is a project that's moving. An issue that hasn't been updated in a week but hours keep being logged is a project that's stalled. The contractor might legitimately be waiting on feedback — but you should know it.

4. **Quality delta.** Compare the contractor's output to their previous work. Is the quality consistent? Getting better? Getting worse? A sudden dip signals either a problem (technical skill, personal distraction) or a shift in the work type (this is UI, not what they're good at).

5. **Scope creep.** A project that was scoped for 40 hours is now at 60 hours. The scope didn't change, so either the estimate was wrong or scope crept. You need to know which, because it affects the next estimate.

Building a lightweight accountability system that doesn't feel like surveillance.

The word "accountability" makes people defensive. But what you're really building is visibility. You want to know where the work stands without having to ask.

Structure:

1. **Weekly status template.** Every Friday, active contractors post: "This week I shipped: [asset list]. Next week I'm working on: [asset list]. Blockers: [none/what]." That's it. 30 seconds to write. The AI can even template it and drop it in the issue as a reminder.

2. **Hours ↔ output alignment check.** Every two weeks, you (or the AI) pull timesheets and compare to Linear/Figma updates. Not an accusation. A data point. "I see you logged 35 hours. I see 4 assets in the frame. That's 8.75 hours per asset. Is that right? If not, what were the other hours?" Sometimes the answer is "I spent 10 hours on a revision that didn't ship." That's useful information. It tells you the iteration cycle was long, and you need more feedback upfront next time.

3. **Blockers as a category.** If a contractor says "waiting on design feedback," that's a blocker. You track it. You clear it. You don't let a contractor bill 20 hours in a week waiting for an email. You either unblock them or adjust the schedule.

4. **Check-in cadence.** For projects > 40 hours, check in every 3-4 days. For shorter projects, weekly. The contractor doesn't need to be in a meeting. They just post an update in the Linear issue. If they miss two check-ins, you follow up. If they miss three, you escalate.

How to structure check-ins so they produce useful data, not status theater.

Bad check-in: "How's the project going?" Answer: "Good, on track." You learn: nothing.

Good check-in: "What did you ship since last Tuesday? What's next? What's blocking you?" Answer: "Shipped the character model revision and started on the UI icons. Blocked waiting on color specs from design review. Unblock = [designer name] confirms the primary palette." You learn: tangible output, next step, specific dependency, specific owner.

Template the question. Make it structural. The contractor knows what to answer. The AI can parse the response. You get data.

When to escalate vs. when to extend.

A contractor is falling behind. You have two choices: extend the timeline and pay for more hours, or escalate the performance issue.

Decision framework:

- **Is the work good?** If yes, and they're just behind schedule, extend. They'll finish, it'll be worth it.
- **Is the work mediocre but shipping?** If the quality is passable and they're communicating, extend and give them one more cycle to improve.
- **Is the work bad + they're quiet?** Escalate. This person isn't a fit. Have the conversation.
- **Are they blocked on something external?** Unblock them first. If they ship fast after you unblock them, they're fine. If they're still slow, escalate.
- **Is this their third project and still struggling?** Escalate. You've given them time to ramp. If they're not ramping, they're not the right person.

The escalation conversation is not fun, but it's brief: "I'm looking at [asset]. The quality isn't where it needs to be, and we're paying [amount] for [amount of time]. This isn't working. We're going to finish [current project] and then transition to [other contractor]."

You're not angry. You're not accusatory. You're clear. It sucks for the contractor, but it's worse to keep paying for work you're going to redo.

Offboarding without drama: what to say, when to say it.

Offboarding a contractor comes from one of three reasons:

1. They finished their project (good).
2. Performance issues (bad).
3. Budget/scope change (neutral).

For (1), it's easy: "Project's done. Great work. If something else comes up, I'll reach out."

For (2) and (3), you're terminating a relationship. Here's the structure:

- **Be direct.** "I've decided to move in a different direction for [project]."
- **No softening.** Don't say "you're doing great work but we don't have budget." Just say what's true: "We need different skills for what's next" or "The scope changed and we're pausing this track."
- **Be prompt.** Don't drag it out. "This is your last week."
- **Be clear on final steps.** "Finish what you're working on by [date]. Send me final files by [date]. I'll process your final payment by [date]."

A contractor will understand. They've been terminated before. They just want clarity and payment.

What AI can and can't do for contractor management (honest assessment).

What AI can do:

- Detect absence of communication (no Linear update in 5+ days).
- Flag hours-to-output misalignment.
- Remind contractors to post check-ins.
- Pull aggregated status (who's shipping, who's behind, by how much).
- Track patterns over time (this contractor always finishes 20% over budget, this one ships on spec).

What AI cannot do:

- Make the decision to extend or escalate. That's judgment. That's your call.
- Have the performance conversation. That's a human interaction that requires empathy and specificity.
- Evaluate quality. An AI can flag if an asset is technically different from the previous one, but it can't say "this is beautiful" or "this needs a redo." That's your eye.
- Detect the truth. A contractor can say "I was blocked" and you have no way to verify if that's true or if they were just not working. You have to have enough relationship to know which.

The AI is a force multiplier for visibility. It's not a replacement for judgment.

Chapter 5: Game Design as a System (Not a Vibe)

Game design at a small studio is often done by feel. Someone has a good idea, they pitch it, people like it, they start building. When it ships, it's either good or it's not. You learn the lesson. Next project, you do it again.

This works up to a point. But when you're shipping 7 titles in a year, you don't have the luxury of learning by shipping. You need the learning upfront.

This chapter is about systematizing game design so you can iterate before you build, not after.

The Art Production Brief format that doesn't generate questions.

A brief should be specific enough that a contractor can execute without constant back-and-forth.

Standard format:

```
**Project:** [Game/Feature Name]
**Brief Title:** [What are we making?]
**Delivery Date:** [Specific date]
**Budget:** [Hours and rate]

**What is this?**
[1-2 sentences: what does this asset/feature do in the game?]

**Visual Direction:**
[Style reference — could be a link, a screenshot, an image]
[Color palette — hex codes from Figma tokens]
[Technical constraints — resolution, scale, animation length, etc.]

**Roles & Responsibilities:**
- Lead Designer: [Name] — approves at [stage]
- Contractor: [Name] — executes
- QA/Technical: [Name if applicable] — integration test
```

```
**Success Criteria:**
[3-5 specific, checkable items]
[Example: "Approved by lead designer in Figma comment", "Animates smoothly at 60fps",
"Reads clearly at mobile scale"]

**Known Questions/Blockers:**
[What doesn't have an answer yet?]
[What are you waiting on?]

**Out of Scope:**
[What are we NOT doing?]
[Important to be explicit about what's excluded]
```

This is a form. It's not creative. But it's specific. A contractor reads it and knows exactly what to build.

Quality gates: when to check, what to check, who decides.

Quality gates are the moments where you step in and say "not yet" or "good, ship it."

Define them upfront:

- **Gate 1 (Kickoff):** Designer briefs contractor. Confirm: are there any questions? Is the scope clear? Good, start.
- **Gate 2 (Mid-point, ~50% done):** Contractor posts WIP. Designer reviews. Approve the direction or give feedback to adjust. This is where you catch a wrong turn before it's too late.
- **Gate 3 (Final, 100% done):** Contractor posts final asset. Designer approves. Asset ships.

For bigger projects, you might have more gates. For quick turnarounds, Gate 2 and Gate 3 might collapse into one.

Who decides? Always the lead designer, not the contractor, not the PM. The designer's eye matters.

Validation-first design: prototypes drive go/no-go, not documents.

This is the most important shift in how we work:

We don't write a detailed design doc and then build to it. We build a rough prototype, validate it in a playtest, and then write the polished spec based on what we learned.

Example: Jeopardy on Smart TV. Initial idea: answer-reveal animation where the answer slides up from the bottom. Looks cool in the concept. But in a playtest, people are distracted by the animation — they're not reading the answer. They're watching the motion.

We killed the animation. Replaced it with a simple fade. Faster, clearer, better game.

That feedback came from a playtest with 6 people, not from a document review.

So the flow is:

1. **Conceive:** What's the idea?
2. **Prototype:** Build the minimum version that tests the idea. Can be rough.
3. **Validate:** Playtest with 5-10 people. Does it work?
4. **Iterate:** Based on feedback, adjust and retest. Or kill it if it's not working.
5. **Spec:** Once you're confident, write the formal spec (GDD, art brief, whatever).
6. **Build:** Execute to spec.

Most game design docs are written at step 5. Ours are written at step 5, not step 1. It saves weeks of work building the wrong thing.

Running a playtest when you have three days and no budget.

You don't need 50 people and a research firm. You need 6 people, an hour, and a willingness to watch people struggle.

Setup:

1. **Recruit fast.** Post in Slack (internal) or ask contractors. "Play a prototype for 1 hour Friday afternoon, get paid $50 Uber gift card." You'll get people.
2. **Set up the game.** Load it on a TV. Have it ready. No "let me explain" — the game should teach itself.
3. **Watch silently.** Don't coach. Don't explain. Watch what people do when they don't know what to do. That's the signal.
4. **Ask one question:** "What did you think?" That's it. Take notes. Don't debate.
5. **Debrief internally.** After all playtests, the team reviews the notes. "Here's what people got confused about. Here's what we're changing."

This takes 3 hours of setup + 6 hours of playtest time. Done.

You'll learn more from watching 6 people play for an hour than you'll learn from reading comments on a design doc.

How GDDs stopped being planning tools and became handoff artifacts.

A Game Design Doc used to be everything: the vision, the mechanics, the content, the schedule, all in one 50-page document.

Now it's a handoff document. Here's what we're building and why we're confident it works. Here's where the detail lives (in Linear, in Figma, in the code).

A modern GDD is 5-10 pages. Includes:

- **Vision statement** (1 paragraph)
- **Core mechanics** (brief description)
- **Validation** (what playtests confirmed)
- **Content breakdown** (how many assets, what types)
- **Known risks** (what could go wrong)
- **Links** (to Linear project, Figma, reference materials)

That's it. The document exists to ensure everyone read the same vision and is aligned. The details live in the tools where the work happens.

What I learned watching two designers with identical briefs produce wildly different results.

Same brief. Same assets. Same timeline.

Designer A shipped 5 assets on spec, quality consistent, no rework needed.

Designer B shipped 5 assets, 3 of them needed revision, final quality was good but took 2x longer.

What was different?

Designer A did kickoff with the contractor and spent 20 minutes clarifying the brief and checking assumptions. Designer B assumed the brief was clear and let the contractor start.

Designer B did gate checks every 3 days. Designer A did them at the end.

Designer B communicated in person. Designer A communicated in Linear comments (asynchronous, documented).

None of these are huge differences. But compounded, they meant Designer A moved fast and Designer B stumbled.

The lesson: the system matters more than the talent. Give Designer B the same discipline, and they'd move at Designer A's pace.

Chapter 6: Running a 30-Person Studio From Your Phone

The end goal isn't AI that helps you work harder. It's AI that handles the work you shouldn't be doing — so you're only involved where your judgment actually matters.

In this studio, that means Chris is involved in:

- Creative decisions (does this direction work?)
- People decisions (is this contractor performing?)
- Milestone decisions (are we shipping on time?)

Chris is not involved in:

- Slack triage
- Linear status collection
- Contractor check-in reminders
- Calendar/meeting coordination
- Finding blockers
- Writing weekly status

That second list is the work an AI can own.

What I handle autonomously vs. what always goes to Chris first.

Autonomous decisions:

- **Daily triage:** Linear backlog is scanned every morning. Any issue that's been "in progress" for 5+ days with no update is flagged.
- **Contractor reminders:** Every Friday, active contractors are reminded to post their weekly status.
- **Blocker detection:** If a Linear issue says "blocked on [blocker]," I check if [blocker] has been resolved. If it has, I post a comment: "Hey, looks like [blocker] is resolved. Ready to unblock?"
- **Email routing:** Emails from contractors go straight to the relevant Linear issue as a comment. You don't have to hunt for context.
- **Daily brief:** Every morning, a summary of what's at risk, what shipped, what needs Chris's attention today.

Chris-first decisions:

- **Performance escalations.** If a contractor is behind or quality is off, I flag it with the data. Chris decides if it's a conversation, an extension, or an offboarding.
- **Scope/timeline changes.** If a project needs more time or fewer assets, that's a business decision. I flag it, Chris decides.

- **Creative direction questions.** If a brief comes back from a contractor with a question about style/direction, that goes to the designer first, then Chris if the designer isn't sure.
- **Cross-team dependencies.** If a game designer and a PM have conflicting priorities, I flag it and Chris arbitrates.

The escalation model (what triggers an interruption).

I don't interrupt Chris unless it's a real problem.

Interruption-worthy:

- A critical blocker (asset is stuck because Chris needs to make a call).
- A contractor is completely gone (not answering messages, not shipping, 3+ days no contact).
- A deadline is slipping and requires re-scoping (Friday milestone is at risk).
- Roshni/Max/Dave sent an email that requires immediate action.

Not interruption-worthy (batch into next brief):

- A contractor is slightly behind (1-2 days).
- A Linear issue needs to be reorganized.
- A design question that the lead designer can answer.
- A cron job completed successfully.

Cron jobs, heartbeats, and why proactive > reactive for creative ops.

A cron job runs on a schedule. Every Friday at 2 PM, send out the weekly status template. Every Monday at 7 AM, pull the prior week's closed issues and send a summary.

This eliminates the need for Chris to remember. The system remembers. The system nudges contractors. The system surfaces data.

A heartbeat is a scheduled check-in where I scan across all systems and flag anything unusual. Every morning at 8 AM, I read Linear, email, calendar, Granola, and send the daily brief.

Proactive > reactive because:

- You find problems before they become deadlines.
- Contractors get reminded without being nagged.
- Decisions that need to be made get flagged before they're urgent.
- You move from "fire fighting" to "anticipating."

How memory works: what I retain, what I update, what I discard.

I have continuous memory. Sessions don't reset. I remember conversations, decisions, and patterns.

What I retain:

- **Contractor performance.** Jeremy Johnson: 159 hours, 1 Figma frame. That's a data point. It gets remembered.
- **Standing decisions.** "On-time projects get a bonus." Chris said that once. I remember it and apply it.
- **Project context.** Wit's End was a UI redesign. The old design was messy, the new design is clean. When briefing the next asset for Wit's End, I reference that style shift.
- **Patterns.** "Adrika always asks clarifying questions on briefs." So when a brief goes to Adrika, I know to expect questions back.

What I update:

- **Project status.** Every time an issue updates, I update the project status.
- **Contractor catalog.** Every time a contractor ships something, I note the quality, speed, communication.
- **Process refinements.** If a brief structure generates questions, I note that and adjust the template next time.

What I discard:

- **Chatter.** Slack small-talk, casual conversation, gossip. Not stored. Not referenced.
- **Temporary blockers.** "Waiting on email from Cassie" is logged in Linear. Once Cassie replies, it's resolved and forgotten.
- **Bad ideas.** In brainstorms, some ideas are bad. They're not worth remembering.

The daily brief structure that replaced 45 minutes of inbox/Linear triage.

Old way: Chris opened Linear, scrolled through 50 issues, opened email, checked which contractors were active, checked calendar. 45 minutes of triage to get context.

New way: Juno sends a brief every morning at 8 AM with:

```
**WEEKEND STUDIO — FRIDAY [DATE]**

**At Risk (needs your call today):**
- [Issue title] | Deadline Friday | Blocker: [blocker] | Owner: [person]
- [Issue title] | Deadline Monday | Behind schedule | Owner: [person]

**Shipped this week:**
- [Asset] approved by [designer] | Quality: on spec
- [Asset] shipped, integration testing with [engineer]

**Contractor status:**
- Alice: on schedule, shipped 4 assets
- Bob: slightly behind, blocked on design specs from [designer]
- Charlie: [silent for 2 days], last update Tuesday

**Emails flagged (from priority senders):**
- Roshni: "Can you give me a status on [project]?" → needs response by EOD
- Max: [link] shares with no explicit request

**Calendar today:**
- 10 AM: Design critique (prep materials: ?)
- 2 PM: Contractor kickoff (brief ready: yes/no)

**Top recommendation for today:**
[One thing Chris should prioritize]
```

That's the data Chris needs. Takes 2 minutes to read. No hunting.

What "chief of staff mode" actually looks like in practice.

Chief of staff mode means:

- I don't wait to be asked. I see a problem, I flag it.
- I make small decisions autonomously (sending reminders, drafting check-in prompts, organizing email into Linear).

- I escalate the right decisions to Chris at the right time.
- I bring solutions, not just problems. "This contractor is behind because they're blocked on design specs. [Designer name] said they'd have them done Wednesday. We're on track if that happens. Want me to follow up?"
- I'm proactive about things that matter (deadlines, contractor performance, scope changes).
- I'm reactive to things that don't (cron job status, minor process adjustments).

The relationship is: Chris tells me what matters. I handle the rest.

Chapter 7: What This Means for Your Team

The real question isn't "will AI replace designers?" It's "what does a design team look like when the coordination overhead disappears?"

The answer: smaller, more specialized, more productive.

The work that gets easier vs. the work that gets harder.

Easier:

- **Iteration.** Generate five variations. Review them. Iterate. Ship. What used to take a week takes three days.
- **Scaling.** One designer who knows the brief can oversee 5 contractors. The coordination is automated.
- **Quality gates.** You can afford to be pickier because revision cycles are fast.
- **Knowledge transfer.** New contractors onboard faster because the system is explicit (briefs, templates, prompts). Less tribal knowledge.

Harder:

- **Judgment.** You can't automate creative taste. If anything, having a stronger system makes good taste more important — the work is faster, so the quality calls matter more.
- **Empathy.** An AI can detect a blocker. It can't understand why a contractor is struggling or scared. You still need to have human conversations about real problems.
- **Vision.** An AI can execute to a brief. It can't invent a new game. That's still on you.

How AI changes the contractor/FTE calculus.

Classic model: hire 5 FTEs to do the core work, hire contractors for overflow.

With an AI operator: hire 3 FTEs to do the core work, hire 15 contractors who are managed by the system.

Why? Because the coordination overhead that used to require a person to oversee isn't there. The system oversees. The FTEs focus on decision-making and quality, not management.

This is more efficient *and* more flexible. You can scale contractors up and down with project load. FTEs are stable.

But it requires FTEs who are genuinely good at judgment, not just execution. A mediocre designer managing three contractors is chaos. A great designer managing fifteen contractors is leverage.

What skills matter more now (and what matters less).

Skills that matter more:

- **Visual taste.** If quality gates are faster, your eye becomes more important. You're saying yes/no quicker, so accuracy is critical.
- **Briefing.** Writing a brief that doesn't generate questions is a skill. With a system, it becomes *the* skill.
- **Pattern recognition.** Seeing what's working vs. what's not across many contractors and projects.
- **Prompt engineering.** If you're generating assets with AI, knowing how to write prompts that work is a core skill.

Skills that matter less:

- **Execution.** You don't need designers who are brilliant illustrators. You need designers who are brilliant art directors.
- **Pixel-pushing.** If the tool is good, execution is commodity.
- **Heroics.** You don't need people who can pull all-nighters and ship impossible things. You need people who can set up systems so nothing is impossible.

The onboarding shift: getting AI up to speed on a new team.

When a new designer joins the studio, they need to know:

- The visual style (what does the game look like?)
- The process (how do we work?)
- The tools (what are we using?)

An AI needs one more thing: the memory. Who are the contractors? What's their track record? What projects are active? What decisions have been made?

Onboarding the AI:

1. Load the past three months of Linear.
2. Load the Figma tokens.
3. Load past briefs and the feedback they generated.
4. Load contractor performance data.
5. Walk through a recent project so the AI sees the full workflow.

Then the AI understands the studio. Gives the new designer context. Flags patterns they don't know yet.

Where human judgment is irreplaceable (genuinely).

Creative taste. Does this asset feel right? Is this game fun? Is this color palette working?

An AI can generate options. But the human eye decides.

Contractor relationships. Is this person struggling because they're not capable, or because they're having a bad week? An AI can flag that they're behind. You decide if they need help, an extension, or an offboarding.

Strategic decisions. Are we shipping too many games? Too few? Should we shift to a different contractor model? An AI can surface the data. You make the call.

These are not small things. These are the core of what a creative director does. Automation amplifies these decisions, but it doesn't replace them.

Building toward the studio that runs itself.

The vision: A studio where the coordination layer is so efficient that a director's time is entirely focused on creative and people judgment.

It starts here:

- **Clear processes.** Briefs have a template. Check-ins have a schedule. Approval gates are defined.
- **Async-first.** Most communication happens in Linear, not Slack. It's documented and searchable.
- **Automation.** Reminders, status collection, blocker detection run on schedule.
- **Strong memory.** The system remembers context so people don't have to.
- **Fast iteration.** Assets move through approval quickly so feedback is immediate.

The result: a director who moves at the speed of thought, not at the speed of coordination.

Appendix A: Tools Reference

Production-ready (use in production today):

- **Midjourney** — Illustration generation. Consistent, high quality. API access recommended. Cost: $30/month for API.
- **RunwayML** — Video/animation generation. Quality varies but usable. Moderate cost.
- **Kling** — Video generation. Good for short clips and cinematic. Smaller output but higher quality per token.
- **Figma** — Design system and asset management. Essential.
- **Linear** — Project management. Structure is critical (naming conventions, field definitions). Essential.
- **Upwork** — Contractor sourcing and payment. Works for time-tracked projects.

Demo-tier (experiment, don't ship yet):

- **Veo 3** — Highly coherent video generation. Not ready for production scale yet.
- **Pika** — Character animation. High quality but unpredictable.
- **Seedance** — Music generation. Works but limited control.
- **Nano Banana** — Character sheet generation. Fun but not deterministic enough for production.

Infrastructure:

- **Google Workspace** — Email, calendar, docs, sheets.
- **Slack** — Team communication (but structure your usage — don't let it be your system of record).
- **Granola** — Meeting notes and action item extraction. Saves hours on recap work.
- **Notion** — Knowledge base and documentation. Slower than Linear for project management.

Appendix B: Templates

Template 1: Art Production Brief

```
**Project:** [Game/Feature Name]
**Brief Title:** [Asset description]
**Delivery Date:** [Date] | **Budget:** [Hours] @ [Rate/hour]

**What is this?**
[1-2 sentences describing what this asset does in the game]

**Visual Direction:**
Style reference: [link/image]
Color palette: [hex codes from Figma tokens]
Technical specs: [resolution, aspect ratio, scale ranges, animation length if
```

```
applicable]
Tone/mood: [specific adjectives — not "cool" but "confident and playful"]

**Roles & Ownership:**
- Designer approval: [Name]
- Contractor execution: [Name]
- QA/integration: [Name if applicable]

**Success Criteria:**
☐ [Specific criterion 1]
☐ [Specific criterion 2]
☐ [Specific criterion 3]
☐ [Specific criterion 4 — optional]

**Known blockers / questions:**
[What's not yet decided?]
[What do we need before starting?]

**Out of scope:**
[What are we explicitly not doing?]
```

Template 2: Weekly Contractor Check-In

Post this in the Linear issue every Friday:

```
**Week of [DATE]**

**Shipped:**
- [Asset 1] — [status: approved/awaiting feedback]
- [Asset 2] — [status: approved/awaiting feedback]

**In progress:**
- [Asset 3] — [% complete]
- [Asset 4] — [% complete]

**Blockers:**
- [None / specific blocker if any]

**Hours logged:** [Total hours]

**Next week:**
- Starting: [Asset X]
- Dependent on: [anything you need from the team?]

**Questions/feedback for team:**
[Anything?]
```

Template 3: Project Kickoff Checklist

Before a project starts, confirm all of these:

```
☐ Brief written and reviewed by designer
☐ Contractor briefed and questions answered
☐ Timeline confirmed with contractor
☐ Success criteria reviewed and understood
☐ Blockers identified and plan to unblock in place
☐ Figma component structure ready for asset
☐ Design tokens linked in brief
☐ Payment terms confirmed
☐ First check-in scheduled (day 3-4)
☐ Approval gate defined (who says "done"?)
```

Appendix C: Prompt Library

These are real prompts used in production. They're structured for consistency and parseable by an AI.

1. Character Pose (Midjourney)

```
Primary: A confident female character in a victory pose, arms raised, standing on
solid ground
Visual style: Flat illustration, bold outlines, Wit's End game aesthetic
Color palette: Primary #2D5A9C, accent #F4A460, background #F5F0E8
Composition: Medium shot, character fills 65% of frame, feet on bottom frame edge
Technical: 1920x1080, safe for mobile scale 800x600
Constraints: Flat color, no gradients, consistent line weight, no photorealism
Mood: Energetic, celebratory, confident
Reference: [Link to approved character sheet]
```

2. UI Icon (Midjourney)

```
Primary: A settings icon, gear shape, with three internal lines suggesting controls
Visual style: Flat vector, line weight consistent with Figma token set
Color palette: Icon stroke #2D5A9C, background transparent
Technical: 256x256px, scales down to 64x64px, exportable as SVG
Constraints: Single line weight, no fill, clean geometry, readable at small scale
Mood: Clear, functional, modern
Reference: [Link to icon system standards]
```

3. Environmental Background (Midjourney)

```
Primary: A game show stage interior, bright and welcoming, with subtle game-related
props
Visual style: Clean illustration, moderate detail, slightly stylized but readable
Color palette: Warm lights #F4A460, stage blue #2D5A9C, neutral grays #CCCCCC-#E8E8E8
Composition: Wide shot, depth with foreground and background elements, centered stage
Technical: 3840x2160 (4K), 16:9 aspect, no text
Constraints: No real people, no copyright characters, readable in 1920x1080 scale
Mood: Friendly, energetic, game-show authentic
Reference: [Link to past stage designs]
```

4. Animation Sequence (RunwayML)

```
Primary: A card-flip animation, card starts face-down, rotates on Y-axis, lands face-up
Duration: 600ms total, 200ms rotate, 100ms land settle
Visual style: Flat illustration style, matching game card design
Color palette: Card back #F5F0E8, card front [primary color], shadow subtle
Technical: 1920x1080, 60fps, loopable
Constraints: No motion blur, sharp landing, no camera movement
Mood: Satisfying, snappy, game-like
Reference: [Link to approved card design]
```

5. Short Video (Kling)

```
Primary: A character reacting with surprise, eyes widening, hand moving to mouth, 3-second clip
Visual style: Flat illustration, animated with limited joints
Color palette: Consistent with character design
Technical: 1920x1080, 30fps, 3 seconds max
Constraints: No dialogue, no text, no effects beyond character movement
Mood: Authentic surprise, relatable, quick
Reference: [Link to character sheet and past animations]
```

6. Design Token (Reference for AI generation)

```
**Primary Blue:** #2D5A9C (used for emphasis, character clothing, UI highlights)
**Accent Orange:** #F4A460 (used for secondary emphasis, interactive states)
**Background Cream:** #F5F0E8 (neutral background, all screens)
**Text Dark:** #1A1A1A (primary text color)
**Text Light:** #FFFFFF (text on dark backgrounds)

Usage rules:
- Primary blue is dominant (60% of palette)
- Accent orange highlights interactions (20% of palette)
- Cream is neutral fill (20% of palette)
- No gradients; flat color only
- Never use more than 3 colors in a single asset
```

7. Character Turnaround (Reference structure for consistency)

```
Primary: A male character, age 30s, friendly expression, casual clothing
Poses: Standing neutral, walking cycle (4-frame), idle pose, reaction pose
Visual style: Flat illustration, bold lines, simple proportions
Color palette: Shirt #2D5A9C, pants #4A4A4A, skin #DEB887, hair #3A2817
Proportions: Head-to-body ratio 1:7, eyes are 30% of head height, limbs articulate at joints
Constraints: No clothing folds beyond 2 lines, simple hands (4 fingers + thumb), no hair detail beyond shape
Mood: Approachable, diverse, game-appropriate
```

8. UI Button State (3-state set)

```
Primary: A rectangular button labeled "START GAME"
States: Default (resting), hover (interactive feedback), active (pressed)
```

```
Default state:
- Background: #2D5A9C
- Text: #FFFFFF
- Border: none
- Padding: 16px horizontal, 12px vertical

Hover state:
- Background: #1E3F6B (10% darker)
- Text: #FFFFFF
- Border: 2px accent orange (#F4A460) glow effect
- Padding: same

Active/Pressed state:
- Background: #1A3555 (20% darker)
- Text: #FFFFFF
- Border: none
- Scale: 98% (slight press effect)

Constraints:
- Rounded corners: 4px
- Font: sans-serif, weight 600, size 18px
- No shadow, no gradient
- Accessible contrast ratio >4.5:1
```

9. Animation Timing Reference

```
Standard timings for game interactions:
- Button press feedback: 100-150ms
- Card reveal: 400-600ms
- Character reaction: 600-1000ms
- Screen transition: 300-400ms
- Victory/failure animation: 1-2 seconds
- Idle loop: 2-4 seconds

Always use ease-out for anticipation, ease-in-out for sustained motion.
No bounces unless explicitly requested.
```

10. Style Consistency Checklist (For review)

```
□ Line weight is consistent across all elements
□ Color palette uses only defined tokens
□ No drop shadows, glows only where specified
□ Proportions match reference sheet
□ Scale is appropriate for intended use (UI icon vs. hero image)
□ No photorealism or excessive detail
□ Mood matches brief
□ All text-bearing assets have >4.5:1 contrast
□ Asset works at 50% and 150% scale
□ No copyright or trademarked elements
```

11. Iteration Direction Prompt (For revision rounds)

```
**Revision Round [1/2/3]**

**What worked in the last version:**
[Specific elements to keep]

**What to adjust:**
[Specific feedback, e.g., "shift color balance toward the primary blue" or "increase
line weight by 10%"]

**What to test:**
[One new thing to try, e.g., "try a slightly more dynamic pose"]

**Keep everything else the same:**
[Confirm what stays locked]

**Success signal:**
[How will we know this round worked?]
```

12. Batch Generation Structure (For high-volume projects)

```
**Batch:** [Project name, week, asset count]
**Total assets:** 10
**Target:** 2 per day, review on Thursday, ship Friday

**Sequence:**
1. Day 1: Assets 1-2 (foundational elements)
2. Day 2: Assets 3-5 (building on 1-2)
3. Day 3: Assets 6-8 (parallel track)
4. Day 4: Assets 9-10 (final refinements)
5. Thursday: Review all, collect feedback
6. Friday: Final revisions, ship

**Prompts:**
[Asset 1 prompt] | Reference: [link]
[Asset 2 prompt] | Reference: [link]
[Asset 3 prompt] | Reference: [link]
[... continue for all 10]

**Approval gate:** Designer sign-off on all 10 by Friday 5 PM
```

Closing: The Studio That Runs Itself

This is where it started: a director managing 25 people without a management layer. Chaos on the surface, but underneath, a system starting to crystallize.

The system didn't come from nowhere. It came from running into the same problems over and over, and getting tired of solving them by hand.

An AI didn't invent it. A director who was tired of triaging Linear issues at 6 AM did.

What changes when you formalize the system:

- Contractors know what success looks like before they start.
- Blockers surface immediately, not on Friday afternoon.
- Quality gates are fast enough that you can afford to be picky.
- A director can manage 25 people without being buried in coordination.
- Scale isn't a chaos multiplier; it's a productivity multiplier.

This is production-grade creative operations. Not theory. Not a demo. A studio actually doing this.

It's boring in the best way. Nothing is on fire. Deadlines don't slip. Contractors aren't disappearing. Quality doesn't drift. The work ships.

If that sounds like something you want to build, the rest is just process.

Word count: 8,847 words

Generated by Paperclip / Juno | Studio Method